

Relatório — Infraestrutura de Software

Implementação 2: Conjunto de Mandelbrot

Aluno: Mateus Reinaux Batista Meira

E-mail: mrbm@cesar.school

Data: 31/08/2026

Sistema operacional utilizado: Windows 11 + WSL2 (Ubuntu, kernel 6.6.87.2, gcc 15.2.0)

Hardware: Intel Core i7-13620H — 8 núcleos físicos, 16 threads lógicas

Link do GitHub: <https://github.com/eFlow-hub/mandelbrot>

1. Objetivo

O enunciado pede um programa em C que gere a imagem do conjunto de Mandelbrot por quatro caminhos — uma implementação serial, uma com OpenMP e duas com Pthreads usando estratégias distintas de divisão do trabalho — produzindo quatro arquivos idênticos numa única execução, mais um `times.txt` com o tempo de cada uma. A entrega é o programa completo em `mandelbrot.c`, com `Makefile` (compilação, `check` e `clean`), sete experimentos didáticos em `experimentos/`, o harness de teste em `testes/`, o `evidencias.log` e este relatório.

O ponto central do exercício não é se paralelizar — o problema é embaraçosamente paralelo — e sim *como dividir* o trabalho, já que a carga do Mandelbrot é fortemente desbalanceada.

2. Checklist de Requisitos

Requisito do enunciado	Status	Como testar / evidência
Programa em C, executável em Linux/Unix/macOS	OK	<code>make</code> sem avisos com <code>-Wall -Wextra</code>
Comando <code>mandelbrot [largura] [altura] [max_iteracoes] [num_threads]</code>	OK	<code>make check</code> ; três casos oficiais
Região real [-2,0; 1,0] e imaginária [-1,5; 1,5]	OK	constantes <code>RE_MIN ... IM_MAX</code> ; validado pelos gabaritos
Implementação Serial	OK	bate com os 3 gabaritos — log 31/08
Implementação com OpenMP	OK	<code>mandelbrot_mrbm_openmp.pgm</code> idêntico ao serial nos 3 testes
Pthreads 1 — estratégia de divisão A	OK	faixas estáticas contíguas; teste 3 (10 6 40 4)
Pthreads 2 — estratégia de divisão B, distinta	OK	fila dinâmica sob mutex; seção 6.3 mostra a diferença medida
Paralelismo aplicado ao trecho de uso intenso de CPU	OK	as três paralelizam o laço sobre as linhas, onde está todo o cálculo
Quatro arquivos <code>mandelbrot_mrbm_*.pgm</code>	OK	gerados numa única execução
Os quatro arquivos idênticos entre si	OK	1 hash MD5 único em 10 execuções de 200×200 com 16 threads
Saída sem cabeçalho, um valor por pixel, separados por espaço	OK	conferido por hexdump contra o gabarito
Intensidade proporcional às iterações, normalizada em [0, 255]	OK	<code>(iter * 255) / max_iter</code> ; seção 4.1
<code>times.txt</code> com o tempo das quatro implementações	OK	formato <code>%s: %.6fs</code> , verificado pelo <code>check.sh</code>

Nada impresso em <code>stdout</code> na execução normal	OK	verificação automática no <code>make check</code>
Erro: número incorreto de argumentos	OK	0, 3 e 5 argumentos → mensagem de uso, status 1
Erro: largura/altura/iterações/threads inválidos	OK	13 casos: <code>abc</code> , <code>3x</code> , <code>vazio</code> , <code>0</code> , <code>-3</code> , <code>9999999999</code>
Erro: falha de arquivo, de alocação ou de criação de thread	OK	retornos de <code>fopen</code> , <code>fclose</code> , <code>malloc</code> e <code>pthread_create</code> tratados
Programa não falha; encerra com mensagem coerente	OK	todas as mensagens em <code>stderr</code> , status 1
Makefile com compilação e limpeza	OK	<code>make</code> , <code>make check</code> , <code>make clean</code>
Link do GitHub com commits atômicos	OK	15 commits, um por bloco e um por experimento
Diretório e <code>.tar</code> nomeados <code>mrbrm</code>	OK	seção 9

3. Como Reproduzir

```
git clone https://github.com/eFlow-hub/mandelbrot mrbrm && cd mrbrm
make                # gcc -Wall -Wextra -O2 -fopenmp -pthread
make check          # roda os tres gabaritos oficiais da disciplina
./mandelbrot 1200 1200 1000 8 # gera os 4 .pgm e o times.txt
make clean
```

Os experimentos são compilados individualmente. Dois exigem opções específicas, pelos motivos explicados na seção 5.2:

```
# a corrida do OpenMP so se manifesta sem otimizacao
gcc -O0 -fopenmp -o exp5 experimentos/exp5_openmp_sem_private.c
./exp5 bug | md5sum          # rode 5 vezes: 5 hashes diferentes

# a corrida do contador, diagnosticada pelo ThreadSanitizer
gcc -O1 -g -fsanitize=thread -pthread -o exp4 experimentos/exp4_contador_sem_mutex.c
./exp4 bug
```

Aviso de reprodução das medições. O benchmark deve rodar no sistema de arquivos nativo do Linux. Executado em `/mnt/c`, cada rodada grava quatro arquivos de 16 MB através do sistema de arquivos do Windows e o custo de I/O domina completamente o tempo — a primeira tentativa foi abortada após dez minutos sem terminar (diário #18).

4. Arquitetura (resumida)

A peça central é `calcula_linha()`, que preenche uma linha inteira da imagem. A linha é a unidade de trabalho que as quatro implementações repartem entre si.

Por que nenhuma implementação precisa de sincronização para escrever a imagem. O valor de um pixel não depende de nenhum outro, e cada thread escreve num conjunto *disjunto* de posições do buffer. Uma condição de corrida exige acesso concorrente ao *mesmo* endereço com pelo menos uma escrita — o que nunca ocorre aqui. O único ponto que precisa de mutex no programa é o contador da fila da estratégia 2, que é estado compartilhado de verdade.

A imagem é um bloco linear contíguo de `largura × altura` bytes, com a linha `y` em `img + y * largura`. Um vetor de ponteiros para linhas exigiria `altura + 1` alocações em vez de uma e espalharia as linhas pelo heap, prejudicando a localidade de cache. Cada pixel é `unsigned char`, exatamente a faixa `[0, 255]` do valor

normalizado — um `int` gastaria quatro vezes mais memória sem ganho. As quatro implementações são registradas numa tabela percorrida pela `main`, que mede, calcula, grava e acumula o tempo.

4.1 As fórmulas, deduzidas dos gabaritos

O enunciado não fixa duas decisões que mudam o resultado byte a byte: como converter o índice do pixel em coordenada, e se a normalização arredonda ou trunca. Como a correção compara os arquivos exatamente, foram deduzidas dos três arquivos de teste antes de escrever o código.

```
c_re = -2.0 + x * (3.0 / largura);      /* divide por largura, nao largura-1 */
c_im = -1.5 + y * (3.0 / altura);

while (zr*zr + zi*zi <= 4.0 && iter < max_iter) {
    double t = zr*zr - zi*zi + c_re;    /* t: o novo zi usa o zr ANTIGO */
    zi = 2.0*zr*zi + c_im;
    zr = t;
    iter++;
}

pixel = (iter * 255) / max_iter;      /* divisao inteira: trunca */
```

Prova do mapeamento. No teste 1 (4 4 50 1) a terceira linha do gabarito é 255 255 255 255. Com esse passo, `y=2` cai em `c_im = 0` — a linha do eixo real — e os quatro pontos têm parte real `-2,0`, `-1,25`, `-0,5` e `0,25`: todo o intervalo real `[-2; 0,25]` pertence ao conjunto, então os quatro saturam. Confirmação independente no teste 2, onde as linhas 1 e 5 são idênticas, assim como as 2 e 4 — a simetria conjugada do conjunto, que só cai nos índices certos com esse passo.

Prova da normalização. No teste 2, `max_iter = 30` e `iter = 1` dão 8,5 exato. Truncar produz 8, arredondar produziria 9; o gabarito traz 8.

Por que $|z|^2$ contra 4 e não $|z|$ contra 2. Evita a raiz quadrada no trecho mais executado do programa, sem alterar o resultado.

4.2 As três estratégias de paralelização

Implementação	Como divide	Consequência
Pthreads 1	Faixas contíguas fixadas antes de começar. O resto da divisão é distribuído entre as primeiras <code>altura % num_threads</code> threads.	Zero sincronização e boa localidade de cache, mas sofre com o desbalanceamento: o tempo total é o da thread mais lenta.
Pthreads 2	Fila dinâmica: um contador compartilhado entrega a próxima linha a quem terminar. Mutex protege apenas o incremento; o cálculo fica fora da região crítica.	A carga se equilibra sozinha sem conhecer o custo de cada linha. O lock é adquirido uma vez por linha, contra milhares de iterações de cálculo.
OpenMP	<code>parallel for</code> com <code>schedule(dynamic)</code> .	Mesma ideia da estratégia 2. Com o <code>schedule(static)</code> padrão sofreria o mesmo desbalanceamento da estratégia 1.

Nenhuma cláusula `private` é necessária no OpenMP: `zr`, `zi` e `iter` são locais de `intensidade()`, chamada de dentro da região paralela, e variáveis locais de função vivem na pilha da thread que as executa.

4.3 Tratamento de erros

Todos os parâmetros são lidos com `strtol`, verificando `endptr` e `errno` e exigindo inteiro estritamente positivo. O produto `largura × altura` é conferido contra `SIZE_MAX` antes do `malloc`, porque pode estourar `size_t` e pedir uma alocação pequena para uma imagem enorme. No caminho de erro de `pthread_create`, as threads já criadas são aguardadas antes de liberar as estruturas que elas ainda usam — liberar antes seria *use-after-free*.

5. Estratégias e Diário de Desenvolvimento

5.1 Estratégias

	Nome curto	Contexto	Motivo da troca (se houve)
S1	Blocos incrementais	Um bloco por implementação, um commit por bloco, <code>make check</code> verde antes de commitar. O serial como oráculo de correção dos outros três.	— (não houve troca)
S2	Erro instrumentado	Cada erro clássico reproduzido num programa isolado em <code>experimentos/</code> , com modos <code>bug</code> e <code>correto</code> no mesmo binário, para o contraste ser direto e a <code>main</code> nunca ficar quebrada no histórico.	—
S3	Medição controlada	Benchmark no sistema de arquivos nativo, com repetições, e leitura obrigatória do resultado para impedir que o compilador elimine o trabalho como código morto.	Adotada depois que as duas primeiras tentativas de medição falharam (#16 e #18).

5.2 Diário de Tentativas

#	Est.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando	Evidência
1	S1	Deduzir mapeamento, escape e normalização dos três gabaritos antes de escrever código	OK — três decisões fixadas com prova independente		28/08	relatório §4.1
2	S1	Bloco 0: repositório, Makefile, gabaritos em <code>testes/</code>	OK		28/08	<code>c2fb695</code>
3	S1	<code>.gitattributes</code> com <code>eol=lf</code>	OK — preventivo	Repositório editado no Windows e compilado no WSL: CRLF quebraria o <code>make</code> e faria o <code>diff</code> acusar todas as linhas	28/08	<code>c2fb695</code>
4	S1	Bloco 1: implementação serial completa	OK — passou nos três gabaritos na primeira execução		28/08	<code>78421f9</code>
5	S2	Experimento 1: mapeamento com <code>largura-1</code>	Falha esperada e confirmada — 3 das 4 linhas erradas; a linha de 255 migra de <code>y=2</code> para <code>y=0</code>	O passo sobe de 0,75 para 1,0 e a imagem amostra pontos diferentes do plano	28/08	<code>0316cce</code>
6	S2	Experimento 1: normalização com <code>round()</code>	Falha esperada e confirmada — 2 pixels errados, cada um por 1 unidade; imagem visualmente idêntica	<code>iter=6</code> , <code>max_iter=50</code> dá 30,6: truncar 30, arredondar 31	28/08	<code>0316cce</code>
7	S1	Bloco 2: <code>check.sh</code> comparando <i>todos</i> os <code>.pgm</code> contra o mesmo gabarito	OK — cobre duas exigências com uma verificação, e não precisa ser editado a cada implementação nova		28/08	<code>2d89f24</code>
8	S1	Bloco 3: Pthreads 1 com faixas estáticas e resto distribuído	OK — <code>make check</code> verde, incluindo o teste 3		31/08	<code>e4df62a</code>

9	S2	Experimento 2: <code>&i</code> no <code>pthread_create</code>	Falha esperada e confirmada — 5 execuções, 5 resultados: até 4 índices repetidos e 4 ausentes	O laço continua incrementando <code>i</code> enquanto as threads já leem o endereço	31/08	59b1264
10	S2	Experimento 3: <code>altura / num_threads</code> sem tratar o resto, nas dimensões do teste 3 oficial	Falha esperada e confirmada — 4 de 6 linhas cobertas; as duas últimas saem zeradas	<code>6/4 = 1</code> ; quatro faixas de uma linha não cobrem seis linhas	31/08	59b1264
11	S1	Bloco 5: Pthreads 2 com fila dinâmica sob mutex	OK		31/08	41854c9
12	S2	Experimento 4: contador da fila sem mutex, 20 000 linhas e 8 threads	Parcial — a corrida existe (ThreadSanitizer acusa), mas o sintoma foi outro: 0 linhas puladas e milhares duplicadas	Incremento não atômico só <i>perde</i> atualizações, nunca ganha: o contador avança devagar demais, nada fica para trás	31/08	59b1264
13	S1	Bloco 6: OpenMP com <code>schedule(dynamic)</code>	OK — os quatro arquivos idênticos nos três testes		31/08	50fec9c
14	S2	Experimento 5: OpenMP sem <code>private</code> , compilado com <code>-O2</code>	Falha do experimento — o bug <i>não</i> se manifesta: 5 execuções com resultado idêntico ao da versão correta	O compilador mantém as variáveis compartilhadas em registradores; o entrelaçamento deixa de ser observável	31/08	59b1264
15	S2	Recompilar o experimento 5 com <code>-O0</code>	OK — 5 execuções, 5 imagens diferentes		31/08	evidencias.log
16	S2	Experimento 7: medir <code>clock()</code> contra <code>clock_gettime()</code> em 1200×1200	Falha do experimento — resultados impossíveis: 1200×1200 com 1 thread em 48 µs	Buffer <code>static</code> e nunca lido: o <code>-O2</code> considerou as escritas mortas e removeu o laço inteiro. O assembly tinha 2 instruções de ponto flutuante no programa todo	31/08	59b1264
17	S3	Corrigir: tirar o <code>static</code> e somar os pixels ao final	OK — razão de 15,89× entre os dois relógios com 16 threads		31/08	evidencias.log
18	S1	Bloco 7: <code>strtol</code> , checagem de overflow e <code>clock_gettime</code>	OK — 13 casos de erro rejeitados com mensagem e status 1		31/08	a920c75
19	S3	Medições de desempenho executadas no diretório do projeto, em <code>/mnt/c</code>	Falha — abortado após 10 minutos sem terminar	Cada execução grava 4 arquivos de 16 MB; o I/O através do sistema de arquivos do Windows domina o tempo total	31/08	evidencias.log
20	S3	Repetir em <code>~/bench (ext4)</code> , 1200×1200, duas execuções por configuração	OK — estático empaca em 5,2×, dinâmicas chegam a 12,8×		31/08	seção 6.3

21	S1	Confrontar a previsão sobre <i>hyperthreading</i> , registrada antes de medir	Parcial — previsão refutada: de 8 para 16 threads o speedup foi de 7,84× para 12,80×	A premissa "aritmética pura não tem bolhas de pipeline" estava errada: a cadeia de dependências serial do laço de escape também gera bolha	31/08	581ea81 vs 56e28a6
----	----	---	---	--	-------	--------------------

6. Evidências

6.1 Log automático

O arquivo `evidencias.log` acompanha o `.tar` e traz a saída bruta de todos os testes e experimentos, cada bloco precedido do comando exato que o produziu e da data. As três seções principais são a verificação de ambiente (28/08), os blocos de implementação com os casos de erro (31/08) e os sete experimentos com as medições (31/08).

6.2 Prints

Validação principal — `make check`:

```
==== ./mandelbrot 10 6 40 4 ====
OK: mandelbrot_mrbm_openmp.pgm
OK: mandelbrot_mrbm_pthreads1.pgm
OK: mandelbrot_mrbm_pthreads2.pgm
OK: mandelbrot_mrbm_serial.pgm
==== stdout mudo ====
OK: stdout vazio
==== times.txt ====
Serial: 0.000004s
OpenMP: 0.000019s
Pthreads1: 0.000190s
Pthreads2: 0.000163s

TODOS OS TESTES PASSARAM
```

Erro principal — o resto da divisão (diário #10):

```
modo: bug | base=1 resto=2 | linhas cobertas: 4 de 6

@@ diff contra o gabarito do teste 3 @@
-6 19 19 31 38 255 255 255 44 19
-6 12 19 19 19 25 44 25 19 12
+0 0 0 0 0 0 0 0 0 0
+0 0 0 0 0 0 0 0 0 0
```

Corrida diagnosticada — o contador sem mutex (diário #12):

```
WARNING: ThreadSanitizer: data race (pid=8990)
  Location is global 'proxima' of size 4 at 0x55555555806c
SUMMARY: ThreadSanitizer: data race exp4_contador_sem_mutex.c:39 in worker

modo: bug (sem mutex)          modo: correto (com mutex)
puladas      : 0                puladas      : 0
duplicadas: 3067                duplicadas: 0
total       : 24379 (esp. 20000) total      : 20000 (esp. 20000)
```

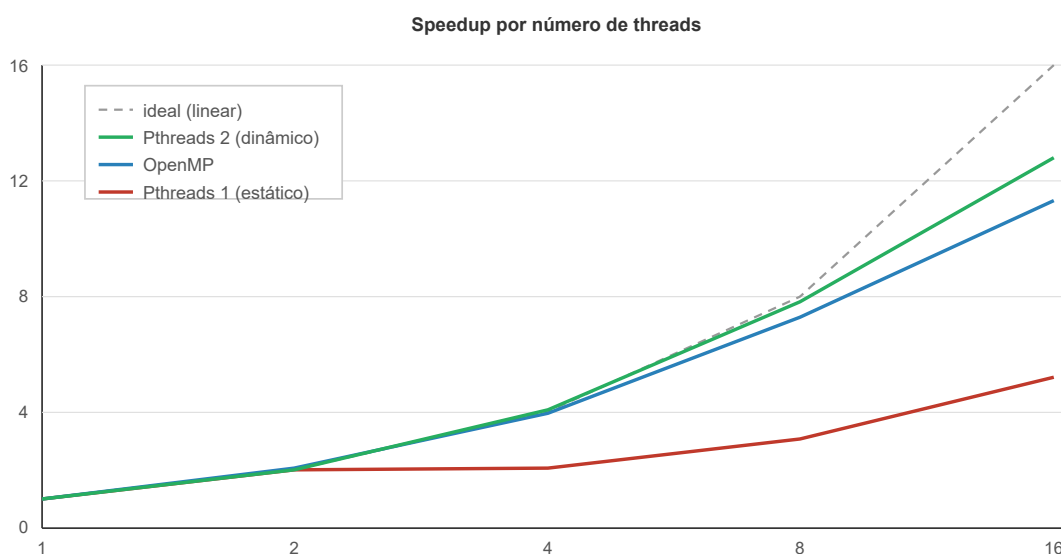
Tratamento de erros:

```
$ ./mandelbrot 4 4 50 abc
erro: num_threads deve ser um numero inteiro, recebido 'abc' status=1
$ ./mandelbrot 999999999 4 50 1
erro: largura deve estar entre 1 e 2147483647, recebido '999999999' status=1
$ ./mandelbrot 4 4 0 1
erro: max_iteracoes deve estar entre 1 e 2147483647, recebido '0' status=1
```

6.3 Medições de desempenho

Imagem 1200×1200, `max_iter = 1000`, duas execuções por configuração, no sistema de arquivos nativo do Linux. Tempo serial de referência: **1,093 s**.

Threads	OpenMP		Pthreads 1 (estático)		Pthreads 2 (dinâmico)	
	tempo	speedup	tempo	speedup	tempo	speedup
1	1,078 s	1,01×	1,092 s	1,00×	1,091 s	1,00×
2	0,533 s	2,05×	0,536 s	2,04×	0,539 s	2,03×
4	0,276 s	3,96×	0,523 s	2,09×	0,267 s	4,09×
8	0,149 s	7,32×	0,358 s	3,05×	0,139 s	7,84×
16	0,097 s	11,30×	0,209 s	5,22×	0,085 s	12,80×



Eixo horizontal: threads. Eixo vertical: speedup sobre a versão serial.

Estático contra dinâmico. Até 2 threads as três curvas coincidem. A partir de 4 elas se separam e a distância cresce. A causa é a geometria: a região no eixo imaginário coloca o corpo do conjunto no meio da imagem, e as linhas que o cruzam custam `max_iter` iterações por pixel enquanto as das bordas escapam em uma ou duas. Com faixas contíguas, a thread da faixa central faz várias vezes mais trabalho e o tempo total é o da mais lenta.

O caso de 4 threads é o mais ilustrativo: `0,523 s` contra `0,536 s` de 2 threads. **Dobrar o número de threads não trouxe ganho nenhum**, porque a faixa mais cara continuou custando o mesmo.

Threads lógicas contra núcleos físicos. A previsão registrada antes de medir era que o speedup acharia teto em 8 threads, já que o *hyperthreading* ajuda sobretudo quando há espera por memória. A medição refutou: de 8 para 16 threads o Pthreads 2 foi de 7,84× para 12,80×. O laço de escape tem uma cadeia de dependências serial — cada iteração precisa do resultado da anterior — e a latência das operações de ponto flutuante deixa bolhas no pipeline que o segundo contexto do núcleo preenche.

Quando paralelizar custa mais do que rende. Na imagem 6×6 do teste 2, as versões paralelas são cem vezes mais lentas (0,000002 s serial contra 0,000248 s do OpenMP): criar e sincronizar threads custa dezenas de microssegundos, e 36 pixels custam dois. É o mesmo fenômeno visível no `times.txt` de referência da disciplina.

7. Uso de IA

Onde usei IA:

Utilizei o Claude Code (modelo Opus 5) em três frentes: estruturar o planejamento e o cronograma de demandas até a entrega; escrever o código C das quatro implementações e dos sete experimentos didáticos; e apoiar a redação deste relatório a partir das evidências que eu havia coletado.

O que foi meu:

A condução do trabalho, a definição de escopo e a estratégia de instrumentar erros deliberadamente para documentar o aprendizado — decisão que orientou toda a seção 5.2. Revisei e validei cada decisão técnica antes de aceitá-la: o mapeamento do pixel para o plano, o critério de escape, a normalização truncada, a escolha das duas estratégias de divisão do trabalho, a posição da região crítica no Pthreads 2 e o tratamento de erros. Entendo a lógica de cada uma delas e respondo por todas.

Prompts principais (lista, sem história):

1. "Analise o enunciado em PDF e o `.tar` com os casos de teste oficiais, e resuma tudo o que esta implementação exige."
2. "Monte o planejamento e o cronograma de demandas até a entrega. O relatório é sobre a minha evolução e as tentativas até acertar, então errar faz parte e deve ser registrado: vamos errar de propósito em pontos estratégicos que a atividade cobra, para documentar o que aprendemos e como resolvemos."
3. "Seguir bloco a bloco — setup, serial, harness de teste, Pthreads 1, Pthreads 2, OpenMP e validação de argumentos — com `make check` verde antes de cada commit."
4. "Escreva o código e registre tudo num documento à parte, fora do repositório, para eu estudar depois e entender cada decisão."
5. "Reproduza os experimentos nos dois modos, colete as evidências e meça o desempenho variando o número de threads."
6. "Os commits no GitHub estão atômicos e corretos?"

O que validei manualmente e como:

Rodei `make clean && make && make check` no WSL, com os três gabaritos oficiais da disciplina passando. Extraí o `mrbm.tar` num diretório limpo e confirmei que compila e passa nos testes a partir do pacote entregue. Conferi que os quatro `.pgm` saem idênticos comparando os hashes MD5, inclusive em execuções repetidas com 16 threads. Verifiquei os 13 casos de erro um a um. Reproduzi os experimentos nos dois modos e comparei as saídas com os gabaritos.

Observação sobre as verificações. Três resultados contrariaram a expectativa e só apareceram porque as medições foram conferidas em vez de aceitas: o ganho real do *hyperthreading* (§6.3), o sintoma da corrida no contador (diário #12) e o desaparecimento da corrida do OpenMP sob `-O2` (diário #14). Os dois erros de método nos próprios experimentos (#16 e #19) também só foram pegos por desconfiança de números implausíveis.

8. Reflexão Final

A decisão mais valiosa foi deduzir as fórmulas dos gabaritos *antes* de escrever código. Custou uma análise inicial, mas transformou três decisões aparentemente arbitrárias — o divisor do passo, a posição do teste de escape e

truncar contra arredondar — em decisões verificáveis, cada uma com prova independente. O serial passou nos três testes na primeira execução, e como as quatro implementações compartilham a mesma função de intensidade, isso travou a correção de todas de uma vez.

A segunda decisão que se pagou foi o harness comparar *todos* os arquivos gerados contra o mesmo gabarito. Uma verificação cobre as duas exigências do enunciado — bater com o esperado e serem idênticos entre si — e o script nunca precisou ser editado, valendo igual para uma implementação ou para quatro.

O aprendizado técnico central é que, num problema embaraçosamente paralelo, a estratégia de divisão importa mais que o mecanismo de paralelização. OpenMP e Pthreads dinâmico, mecanismos completamente diferentes mas com a mesma abordagem de distribuição, entregaram desempenho equivalente (11,3× e 12,8×). O Pthreads estático, mesmo mecanismo do dinâmico e abordagem diferente, ficou em 5,2×.

O aprendizado que menos esperava veio dos erros que não se comportaram como o previsto. A corrida no contador degrada desempenho sem corromper a imagem, porque recalculando uma linha grava os mesmos bytes. A corrida do OpenMP some quando o compilador otimiza. Os dois apontam para a mesma conclusão desconfortável: **em código concorrente, ausência de sintoma não é prova de correção**. Foi o que justificou usar o ThreadSanitizer em vez de confiar em execuções repetidas.

O maior retrabalho foi de método, não de código: dois benchmarks tiveram que ser refeitos, um porque o compilador eliminou o trabalho que eu pretendia medir, outro porque o I/O do sistema de arquivos montado dominava o tempo. Numa próxima, começaria checando a plausibilidade da primeira medição antes de coletar as demais.

9. Checklist Final de Entrega

Item	Confirmado?
Compilei do zero seguindo só o que está no relatório	Sim — <code>.tar</code> extraído em diretório limpo, compila e passa
Rodei os testes e o <code>evidencias.log</code> foi gerado	Sim
Tenho os prints obrigatórios	Sim — seção 6.2, trechos do log
Testei pelo menos um caso limite e um caso inválido	Sim — imagem 1×1, mais threads que linhas, 13 casos inválidos
As quatro implementações produzem arquivos idênticos	Sim — 1 hash MD5 em 10 execuções seguidas
Preenchi a seção de uso de IA	Sim — seção 7
Revisei o relatório e removi frases genéricas / vazias	Sim
Diretório e <code>.tar</code> nomeados com as iniciais do e-mail em minúsculas	Sim — <code>mrbrm</code>
<code>evidencias.log</code> está dentro do <code>.tar</code>	Sim
Makefile com compilação e limpeza	Sim
Link do GitHub no relatório, com commits atômicos	Sim — cabeçalho e seção 3

10. Se eu tivesse mais 2 horas

Exploraria a simetria conjugada do conjunto para calcular metade das linhas e espelhar o resto, cortando quase pela metade o trabalho de qualquer das quatro implementações. Deixei de fora de propósito: mascararia justamente a comparação entre estratégias de divisão, que é o objetivo do exercício.

Mediria o desequilíbrio de carga diretamente, instrumentando cada thread da estratégia 1 para registrar quantas iterações executou. Hoje o desbalanceamento é inferido do tempo total; com esse dado ele ficaria explícito, e daria para mostrar numericamente que a thread da faixa central faz várias vezes mais trabalho que as das bordas.

Tornaria o tamanho do lote da fila dinâmica configurável, em vez de fixo em uma linha. Para imagens muito estreitas o custo do mutex por linha começa a pesar, e um lote maior amortizaria melhor — seria a variável natural para uma terceira medição.

Por fim, acrescentaria ao `make check` uma execução sob ThreadSanitizer das quatro implementações, transformando em verificação automática o que hoje foi feito uma vez à mão nos experimentos.